Geographically Weighted PCA for `spml`

A suggested implementation

PySAL — SDSU spatial-ML group

A 10-minute walk-through of the design, the maths, and what is already on the `gwpca` branch.

The request

Extend the GW framework to unsupervised models, starting with GWPCA.

- Today `spml` fits one *supervised* scikit-learn estimator per location: `GWLinearRegression`, `GWLogisticRegression`, GW random forests, GW gradient boosting.
- All operate on pairs (X, y).
- We want a branch that works on the **feature matrix X alone** and decomposes its *local* covariance structure.
- First estimator: **Geographically Weighted PCA** (Harris, Brunsdon & Charlton, 2011).

Recap: the "map of models" idea

For each location i:

1. Build spatial weights w_{ij} from a **kernel** and a **bandwidth** (fixed distance or adaptive k-NN), via `libpysal.graph`.
2. Fit a local model at i, weighting each observation j by w_{ij}.
3. Store the local parameters for i.

GWPCA keeps steps 1 and 3 unchanged. Only **step 2** is different: no target, no `.fit(X, y)` — a direct eigendecomposition instead.

What GWPCA produces

A principal components analysis whose loadings and variances **vary across space**:

- local loadings — how variables combine into components *here*
- local explained variance — how much structure each component captures *here*
- the **winning variable** — which variable dominates component 1 at each location (the classic GWPCA map)
- proportion of total variation (PTV) retained by the first k components

One global rotation $\Rightarrow$ a surface of rotations.

The one real wrinkle

`sklearn.decomposition.PCA` has no `sample_weight`

We cannot just hand weighted rows to scikit-learn.

So the local model is **not** an sklearn estimator fit — it is a weighted covariance matrix built directly and eigendecomposed with NumPy.

- cheap: one $p \times p$ symmetric eigen-problem per location
- deterministic — no solver, no `random_state`
- the global baseline *is* a plain `PCA`, for comparison

The maths (per location i)

Geographically weighted mean of variable k:

$$\bar{x}_k(i) \;=\; \frac{\sum_j w_{ij}\, x_{jk}}{\sum_j w_{ij}}$$

Geographically weighted covariance:

$$\Sigma_{k\ell}(i) \;=\; \frac{\sum_j w_{ij}\,\big(x_{jk} - \bar{x}_k(i)\big)\big(x_{j\ell} - \bar{x}_\ell(i)\big)}{\sum_j w_{ij}}$$

With `standardize=True` (the default), x is first z-scored **globally**, once, with the population SD:

$$x_{jk} \;\leftarrow\; \frac{x_{jk} - \bar{x}_k}{\mathrm{sd}(x_k)}$$

This is *not* a local correlation matrix: the diagonal of $\Sigma(i)$ is not forced to 1. It only becomes a correlation matrix in the large-bandwidth limit.

The maths — decomposition and outputs

Symmetric eigendecomposition:

$$\Sigma(i) \;=\; V(i)\,\Lambda(i)\,V(i)^{\top}, \qquad \lambda_1(i) \geq \cdots \geq \lambda_p(i)$$

- columns of $V(i)$ — local loadings; $\Lambda(i)$ — local variances
- explained-variance ratio: $\lambda_m(i) / \sum_m \lambda_m(i)$
- score for each neighbour j of i (raw, *not* mean-centered — matches `GWmodel`, more shortly): $s_j(i) = x_j^{\top} V(i)_{[:,\,1:k]}$
- winning variable: $\arg\max_k \left| V_{k1}(i) \right|$

Sign convention: force the largest-magnitude loading of each component positive (per location).

How it fits the existing architecture

Existing piece	**In GWPCA**
`_build_weights`	reuse as-is (kernel graph, adaptive/fixed)
parallel neighbourhood fan-out	reuse (drop the `_y` column)
`_fit_local`	generic; calls `_decompose_local`
prediction helpers	reuse later for `transform`
`keep_models` (mem / disk)	reuse: store local mean + loadings
IC / hat-matrix / AIC code	**skip** (`_model_type = "gwpca"`)
`fit_global_model`	reuse: baseline is a plain `PCA`
`BandwidthSearch`	extend later (CV reconstruction error)

Net new code is small and localised in one module.

New module: spml/decomposition.py

```
from spml.decomposition import GWPCA

gwpca = GWPCA(
    n_components=2,
    bandwidth=30,
    fixed=False,           # adaptive k-NN
    kernel="bisquare",
    standardize=True,      # global z-score + local cov (see R note)
    keep_models=True,      # needed for transform() later
).fit(X, geometry=gdf.representative_point())  # points only
```

- `fit(X, geometry)` — no y; geometry must be points
- Same split as `BaseRegressor` → `GWLinearRegression`: generic machinery in the base class, method-specific code in the subclass (next slide)

Base class vs GWPCA

BaseGWDecomposition

- validation, global standardization
- weights, parallel loop, model storage
- `_fit_local`: weights + local mean, then calls `_decompose_local`
- `_scale_eigenvalues`: divide by each location's *own* weight sum
- generic attributes: `components_`, `explained_variance[_ratio]_`
- `model=` builds the global baseline

GWPCA

- `__init__` with `model=PCA`
- `_decompose_local`: scatter matrix, `eigh`, sign rule, neighbour scores
- `_scale_eigenvalues` override: the GWmodel bug (shared divisor)
- `fit` wrapper: `local_scores_`, `local_loadings_`, PTV, winning variable

The GWmodel bug lives in one method: dropping it means deleting the override.

Fitted attributes (scikit-learn flavoured)

```
gwpca.components_                # (n_samples, n_components, n_features)
gwpca.explained_variance_        # (n_samples, n_components)
gwpca.explained_variance_ratio_  # (n_samples, n_components)
gwpca.local_eigenvalues_         # all eigenvalues, descending
gwpca.proportion_total_var_      # PTV of the first k components
gwpca.winning_variable_          # argmax |loading| on PC1
gwpca.local_scores_              # per-neighbour scores (see R note)
gwpca.local_loadings_            # tidy MultiIndex (location, component)
gwpca.global_model_              # sklearn PCA baseline
```

What is on the gwpca branch now

Implemented (the unambiguous core):

- weighted mean / covariance, global standardization, `eigh`, descending sort, per-location sign convention
- all the attributes on the previous slide
- `graph=` and `geometry+bandwidth` paths; fixed and adaptive; `keep_models`; global baseline
- **19 tests**; rebased on current `main`, full suite green (405 passed, 3 skipped)
- user-guide notebook (Guerry data), `api.rst`, references

Verified against ground truth:

- matches a hand-rolled weighted covariance + `eigh`
- `boxcar` kernel + huge bandwidth reproduces global `sklearn` PCA loadings and ratios to machine precision

Since: code-reviewed and hardened — bandwidth input validation (matching the supervised GW models), dead-code/lint cleanup, vectorized attribute assembly, doc fixes. All six review findings closed.

A wrinkle found while cross-checking against R

Went a step further than the checks on the previous slide: ran real `GWmodel::gwpca` (R) on identical data and diffed the output directly — and found a genuine bug in the reference implementation.

The bug

`gwpca()` normalises every location's reported eigenvalues by `sum(wt)` taken from whichever neighbourhood was *last processed* in the input's row order (a stray/reused loop variable in the R source) — **not** that location's own neighbourhood weight-sum.

- Harmless only when every neighbourhood has the same weight-sum, which in practice means an adaptive `boxcar` kernel.
- Wrong for any distance-decay kernel, **including the default adaptive bisquare** (Guerry, 30 neighbours: weight-sums range from 8.6 to 13.9), and for fixed kernels over irregular points: eigenvalue *magnitudes* are mis-scaled for every location except the one processed last.
- Loadings and `explained_variance_ratio_`/PTV are **unaffected** — the shared divisor cancels out of any within-row ratio and never touches the eigenvectors.

Our call: replicate it, on purpose

`spml` now deliberately reproduces this exact behaviour, for numerical parity with the reference implementation:

- `local_eigenvalues_` / `explained_variance_` share one divisor (the last-processed neighbourhood's weight-sum) $\Rightarrow$ now **depend on `X`'s row order** at fit time.
- Documented in `decomposition.py`'s module docstring ("Deliberate parity with `GWmodel::gwpca`") and locked in by `test_gwpca_eigenvalue_scale_depends_on_row_order`.
- `test_gwpca_matches_r_gwmodel_reference`: eigenvalues, loadings and per-neighbour scores match real `GWmodel::gwpca` output.
- `gwpca()` has no `scale.vars` argument; users call `scale()` first. `standardize=True` does the same with the population SD: same loadings, eigenvalues differ by $(n-1)/n$.
- "Last processed" is looked up by `X`'s last row label, not the spatial graph's internal order.

Worth reporting upstream to the `GWmodel` maintainer.

Two more GWmodel bugs: bandwidth cross-validation

`gwpca.cv` (bandwidth score) and `gwpca.cv.contrib` (per-observation residuals, used for outliers) compute, for each left-out i:

```
v <- v %*% t(v)                                  # projector V_k V_k^T
score <- score + sum((x[i, ] - x[i, ] %*% v))^2
```

- **Uncentred:** reconstructs raw x_i, while the loadings come from data centred on the local mean.
- **`sum(r)^2`, not `sum(r^2)`:** squares the *sum* of the residuals, so positive and negative residuals cancel.
- On Guerry: the textbook score picks **14** neighbours, the GWmodel score **44**.
- Brunsdon's tutorial supplies a "fixed" `gwpca.cv.contrib`, but it has the same bug.

Unlike the eigenvalue bug, this changes *which bandwidth and which outliers* are chosen, so the proposal is a correct score by default, with GWmodel's version as an option.

Open decisions — need the API owners

- **Base-class name** (`BaseGWDecomposition` or `BaseDecomposition`) and whether `model=` stays (it now builds the global baseline).
- **GWmodel eigenvalue bug:** keep it as the default, put it behind an option, or drop it.
- **`local_scores_`:** keep GWmodel's per-neighbour, uncentred scores, or also offer one centred score per location.
- **CV score** for bandwidth selection: textbook or GWmodel.
- **Cross-location sign & order** — eigenvectors are defined up to sign, and component order can swap between neighbours. Per-location convention is in; *aligning across space* (Procrustes rotation to the global solution) is still open — and the usual critique of GWPCA maps.
- **Fixed vs adaptive `n_components`** — fixed for now.

Roadmap

First PR (this branch): `BaseGWDecomposition` + `GWPCA` + core attributes + tests + user-guide notebook, `api.rst` and references. *Code done; PR to `pysal/spml` not yet opened.*

Follow-up PRs:

1. `BandwidthSearch` integration — leave-one-out CV (the notebook computes it by hand for now)
2. Monte Carlo test of local variation
3. cross-location alignment (Procrustes)
4. `transform()` on new data — nearest / ensemble local models (reuse the prediction helpers)
5. `batch_size` for large samples; robust GWPCA and outliers

Get involved

- Branch: `gwpca`; upstream PR to `pysal/spml` in preparation
- Start with the user-guide notebook, `docs/source/gwpca.ipynb`
- Good pickups for the group: bandwidth search (CV), the Monte Carlo test, the alignment question, `transform`

```
from spml.decomposition import GWPCA
```